

Python Portfolio — Architecture Analysis

Project: Python Portfolio
Technology: Python, FastAPI, SQLAlchemy, SQLite, Pydantic, JWT
Location: /home/dominic/Documents/python-portfolio
Report #: 2/20

Architecture Type

Modular unified FastAPI application with 5 domain modules

Layers

Auth Router → Shortener / Blog GraphQL / Chat / Queue / RAG routers →
SQLAlchemy → SQLite

Design Patterns

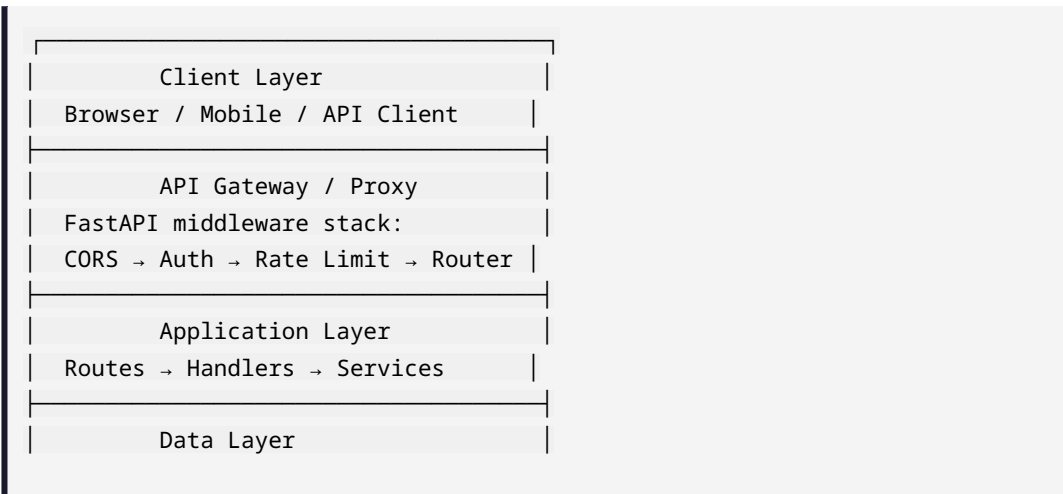
Router-Service pattern, Dependency Injection via FastAPI Depends

Scalability

Single-process SQLite — limited concurrency, suitable for low traffic

System Architecture

Layer Architecture



| SQLAlchemy ORM → SQLite/PostgreSQL|

Component Interaction

Python Portfolio Component Architecture

|— API Layer (FastAPI)

|— Service Layer (Business Logic)

|— Data Layer (SQLAlchemy + SQLite)

|— Client Layer (None)

Request Lifecycle

1. **HTTP Request** arrives at FastAPI

2. **CORS middleware** checks origin headers

3. **Auth dependency** extracts and validates JWT

4. **Rate limiter** checks request count per IP

5. **Router** matches path to handler

6. **Handler** executes business logic

7. **Database session** commits or rolls back

8. **Response** returned to client

Dependency Graph

Python Portfolio Dependency Flow:

Client → FastAPI → Router → Handler → SQLite → Response

Design Patterns Used

Pattern	Location	Purpose
Dependency Injection	FastAPI Depends()	DB sessions, auth
Repository Pattern	SQLAlchemy models	Data abstraction
Middleware Chain	FastAPI middleware	Cross-cutting concerns
Router Module	APIRouter include	Separation of concerns